

<本資料で取り扱う内容>

レンズや鏡があった時に、光はどのように進むのか、像はどこにできるのか、といったことを考えることは幾何光学の入り口となる話です。本資料では、幾何光学の基本原理である反射の法則および屈折の法則のみを用いて、凹面鏡や凸レンズにおける光線追跡を数値的に再現します。計算には Python を用います。

本資料を通して、幾何光学を学んだことがない方、初学者の方、レーザー加工などで光学系に携わる方たちにとって、幾何光学への理解を深める一助となればうれしく思います。

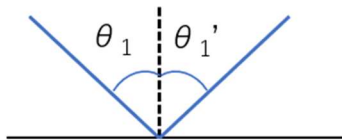
光学の基礎法則

反射の法則

反射光線は入射面内を進み、入射角 θ_1 と θ_1' の間には以下の関係がある。

$$\theta_1 = \theta_1'$$

これを、**反射の法則**という。

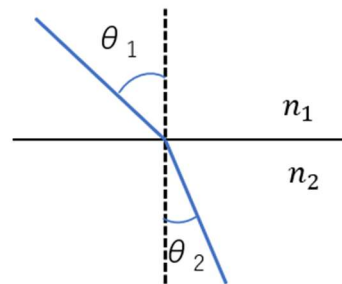


屈折の法則

屈折率 n_1 と n_2 の媒質が平面で接している。屈折率 n_1 の媒質から n_2 の媒質へ光が進むとき、以下の関係を満たす

$$n_1 \sin \theta_1 = n_2 \sin \theta_2$$

これを**屈折の法則**という。



1.凹面鏡

凹面鏡の凹面について、以下のように定義する。

$$\frac{x^2}{l^2} + \frac{y^2}{l^2 - d^2} = 1 \quad (1.1)$$

ここで、 $\alpha = l^2, \beta = l^2 - d^2$ とおく

右図 1.1 のように、点光源 $A(a,b)$ から凹面鏡上の点 $P(p,q)$ を通って反射するときの光路について考える。この時、点 P を通る法線に対して点 A と線対称な点 A' を通る。 A' の座標を (a',b') とおく。

まず、点 P を通る法線を求める。

式(1)を $z = F(x,y) = 0$ と考えると以下が成り立つ。

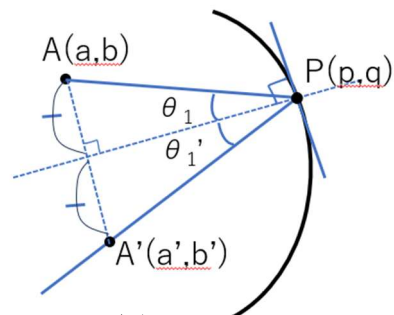


図 1.1

$$dz = dF(x, y) = \frac{\partial F}{\partial x} dx + \frac{\partial F}{\partial y} dy$$

$$= \left(\frac{\partial F}{\partial x}, \frac{\partial F}{\partial y} \right) \cdot (dx, dy) = 0$$

なので、

$$\nabla F \cdot (dx, dy) = 0 \quad (1.2)$$

(dx, dy)が接線ベクトルで、 ∇F との内積がゼロ
ということは、 ∇F が法線ベクトルとなる。

よって、法線ベクトルを求めると、式(1)より、

$$\nabla F = \left(\frac{2x}{\alpha}, \frac{2y}{\beta} \right) = 2 \left(\frac{x}{\alpha}, \frac{y}{\beta} \right) \quad (1.3)$$

(2)より、法線ベクトルは $\left(\frac{p}{\alpha}, \frac{q}{\beta} \right)$ である。

式(3)より、

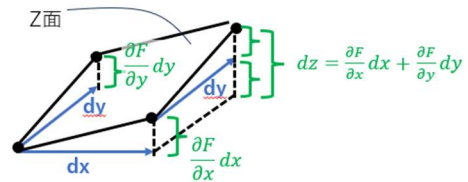
$$n = \left(\frac{p}{\alpha}, \frac{q}{\beta} \right), \quad |n| = \sqrt{\left(\frac{p}{\alpha} \right)^2 + \left(\frac{q}{\beta} \right)^2}$$

$$\hat{n} = \frac{1}{\sqrt{\left(\frac{p}{\alpha} \right)^2 + \left(\frac{q}{\beta} \right)^2}} \left(\frac{p}{\alpha}, \frac{q}{\beta} \right) \quad (1.4)$$

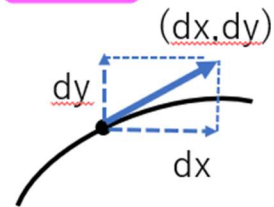
補足

$\frac{\partial F}{\partial x} \cdot dx$: $F(x, y)$ について、x方向に微小距離dxだけ動いたときのz方向の変化量

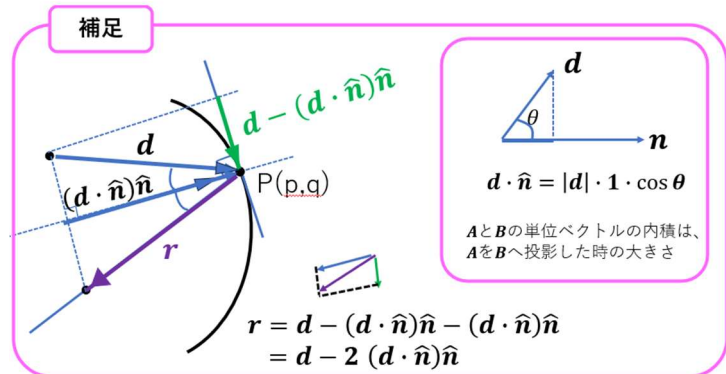
$\frac{\partial F}{\partial y} \cdot dy$: $F(x, y)$ について、y方向に微小距離dyだけ動いたときのz方向の変化量



補足



次に、入射ベクトルと反射ベクトルを求める。入射ベクトル $\mathbf{d}$ とすると、図 1.1 より以下のようなになる。



$$\mathbf{d} = \overrightarrow{AP} = (p - a, q - b) \quad (1.5)$$

反射ベクトルを $\mathbf{r}$ とすると、以下の式で表すことができる。(上記の補足を参照)

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}} \quad (1.6)$$

よって、反射光の直線は媒介変数tを用いて、以下の式で表すことができる。

$$(x, y) = (p, q) + t(r_x, r_y) \quad (1.7)$$

式(1)の l を 8 で固定し、 $d=6,3,0,-3$ にしたときの凹面と入射光と反射光の軌跡を図 1.2(a)~(d)に示す。

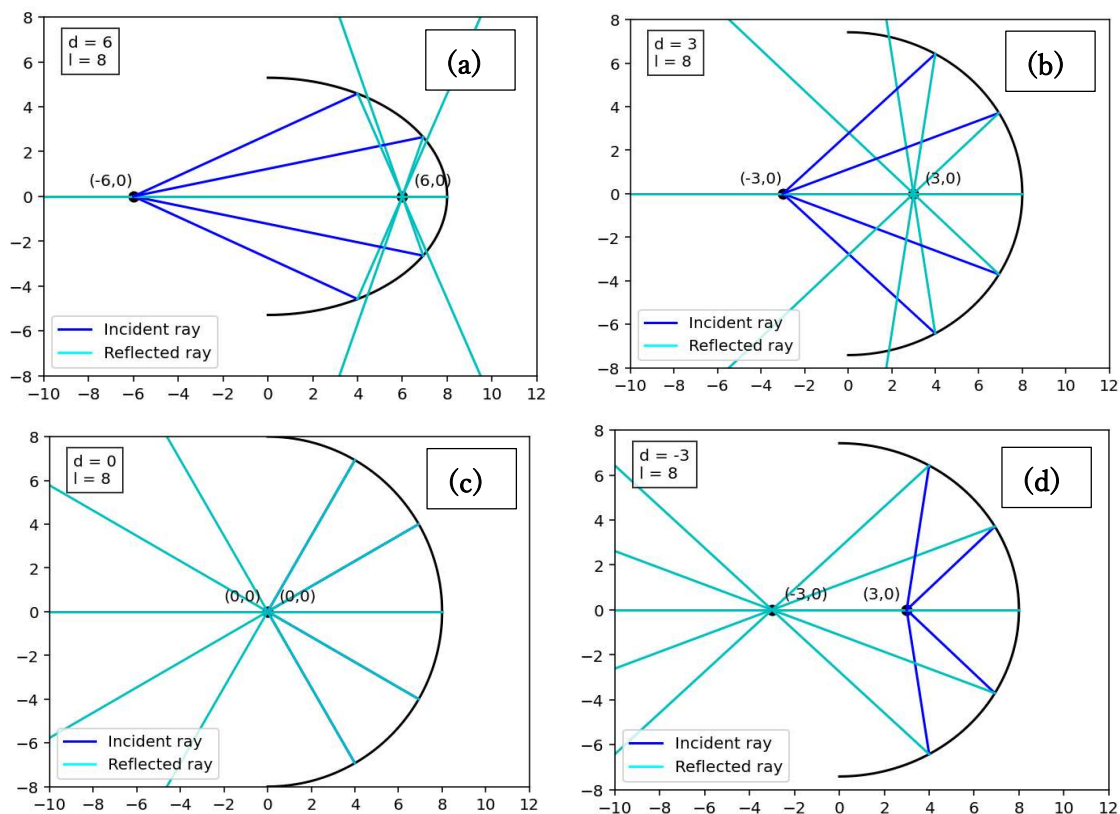


図 1.2 凹面鏡の入射光・反射光

$$\frac{x^2}{l^2} + \frac{y^2}{l^2 - d^2} = 1 \quad (1.1)$$

式(1)で $d=0$ の時は、円となる。よって、 d と l の値の差が大きいほど、上図では横長の楕円となり光源と像の位置は離れており、 d と l の値が近づくほど円に近づき、光源と像の距離が近づくことがわかる。 $d=0$ となった時に円となるため、光源と像の位置が一致する。図 1.3(b)(d)より、 d の符号が逆転すると、光源の位置と像の位置が入れ替わることがわかる。

図 1.3 に点光源を焦点の位置からずらしたときの入射光と反射光の軌跡を示す。

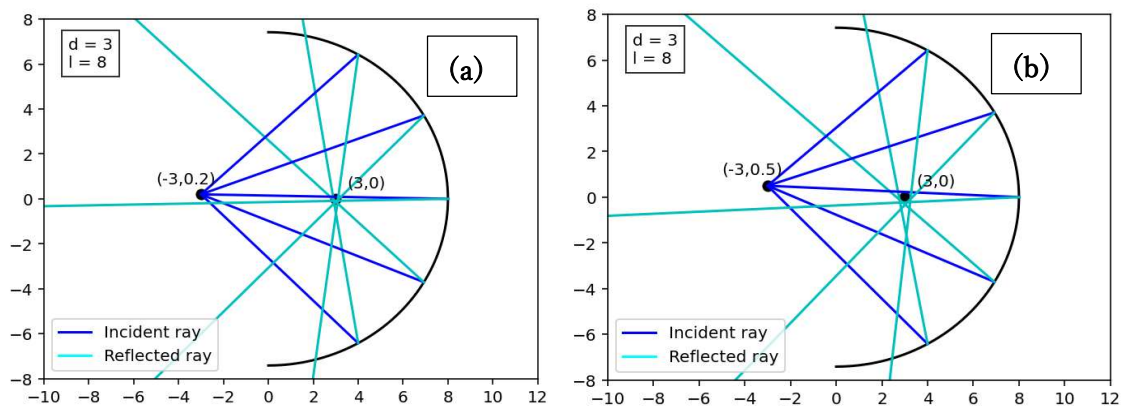


図 1.3 点光源を焦点からずらした場合

図 1.3(a)が点光源を焦点から上方向に 0.2 ずらした場合、図 1.3(b)が点光源を焦点から上方向に 0.5 ずらした場合である。ずれが大きくなるほど、焦点での集光度合いが悪くなっていることがわかる。

計算に用いたプログラムは巻末に示す。

2. 凸レンズ

凸レンズを通る光路を屈折の法則を用いて求めることを考える。凸レンズの光路を考える前に、まず、屈折の法則をベクトルに変換することを考える。

η を媒質の屈折率とすると、屈折の法則は以下の式で表すことができる。

$$\eta_0 \sin \theta_0 = \eta_1 \sin \theta_1 \quad (2.1)$$

入射ベクトル $\mathbf{d}_0$ と法線ベクトル $\mathbf{n}$ を含む面があるとし、その法線ベクトルと直交しているベクトル $\mathbf{v}$ をとし、その単位ベクトルを $\hat{\mathbf{v}}$ とする。法線ベクトルの単位ベクトルを $\hat{\mathbf{n}}$ とすると、屈折ベクトル $\mathbf{d}_1$ は以下のように表すことができる。(図 2.1)

$$\mathbf{d}_1 = -\hat{\mathbf{v}} \sin \theta_1 - \hat{\mathbf{n}} \cos \theta_1 \quad (2.2)$$

入射ベクトル $\mathbf{d}_0$ について、以下の関係が成り立つ。(図 2.2)

$$\begin{aligned} \hat{\mathbf{d}}_0 &= \hat{\mathbf{v}} \sin \theta_0 + (\mathbf{d}_0 \cdot \hat{\mathbf{n}}) \hat{\mathbf{n}} \\ \hat{\mathbf{v}} &= \frac{\mathbf{d}_0 - (\mathbf{d}_0 \cdot \hat{\mathbf{n}}) \hat{\mathbf{n}}}{\sin \theta_0} \end{aligned} \quad (2.3)$$

式(2.3)を(2.2)に代入すると

$$\mathbf{d}_1 = -\frac{\mathbf{d}_0 - (\mathbf{d}_0 \cdot \hat{\mathbf{n}}) \hat{\mathbf{n}}}{\sin \theta_0} \sin \theta_1 - \hat{\mathbf{n}} \cos \theta_1 \quad (2.4)$$

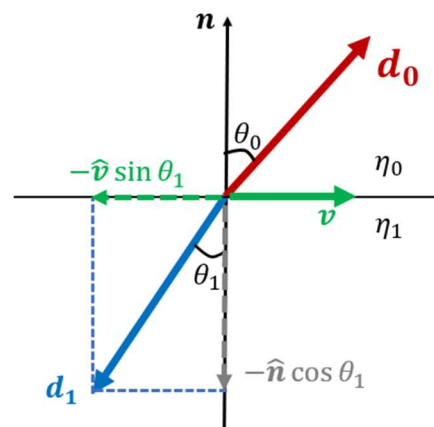


図 2.1

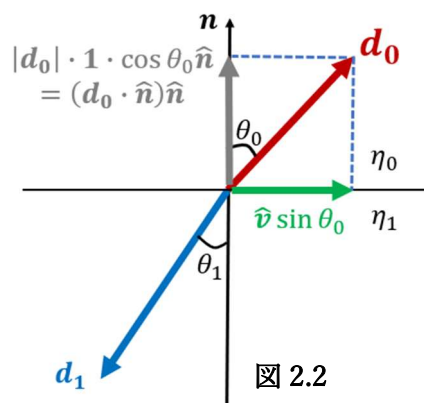


図 2.2

屈折の法則より $\eta_0 \sin \theta_0 = \eta_1 \sin \theta_1$ であるため、 $\sin \theta_0 = \frac{\eta_1 \sin \theta_1}{\eta_0}$ として、 $\sin \theta_1$ にそろえると、

$$\begin{aligned} \mathbf{d}_1 &= -\frac{\mathbf{d}_0 - (\mathbf{d}_0 \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}}{\frac{\eta_1 \sin \theta_1}{\eta_0}} \sin \theta_1 - \hat{\mathbf{n}} \cos \theta_1 \\ &= -\frac{\eta_0}{\eta_1} (\mathbf{d}_0 - (\mathbf{d}_0 \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}) - \hat{\mathbf{n}} \cos \theta_1 \end{aligned} \quad (2.5)$$

$\cos \theta_1$ について変形すると、

$$\mathbf{d}_1 = -\frac{\eta_0}{\eta_1} (\mathbf{d}_0 - (\mathbf{d}_0 \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}) - \hat{\mathbf{n}} \sqrt{1 - \sin^2 \theta_1}$$

$\eta_0 \sin \theta_0 = \eta_1 \sin \theta_1$ であるため、これを θ_0 に揃えると、

$$\begin{aligned} \mathbf{d}_1 &= -\frac{\eta_0}{\eta_1} (\mathbf{d}_0 - (\mathbf{d}_0 \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}) - \hat{\mathbf{n}} \sqrt{1 - \left(\frac{\eta_0}{\eta_1}\right)^2 \sin^2 \theta_0} \\ &= -\frac{\eta_0}{\eta_1} (\mathbf{d}_0 - (\mathbf{d}_0 \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}) - \hat{\mathbf{n}} \sqrt{1 - \left(\frac{\eta_0}{\eta_1}\right)^2 (1 - \cos^2 \theta_0)} \end{aligned} \quad (2.6)$$

$\cos \theta_0$ は、入射ベクトルの単位ベクトル $\widehat{\mathbf{d}}_0$ と法線ベクトル $\hat{\mathbf{n}}$ の単位ベクトルを用いると $\widehat{\mathbf{d}}_0 \cdot \hat{\mathbf{n}}$ で表すことができるので式(2.6)は以下の式で表すことができる。

$$\mathbf{d}_1 = -\frac{\eta_0}{\eta_1} (\mathbf{d}_0 - (\mathbf{d}_0 \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}) - \hat{\mathbf{n}} \sqrt{1 - \left(\frac{\eta_0}{\eta_1}\right)^2 (1 - (\widehat{\mathbf{d}}_0 \cdot \hat{\mathbf{n}})^2)} \quad (2.7)$$

式(2.7)を用いて、凸レンズを通る光路を考える。

凸レンズ面については、図 2.3 のよ

うに、 $(x - a)^2 + y^2 = r^2$ と

$(x - a)^2 + y^2 = r^2$ によってできるものであると考える。

点光源の座標を $P_0(p_0, q_0)$ とする。入射光が凸レンズ上の点 $P_1(p_1, q_1)$ に到達したとき、入射ベクトル $\mathbf{d}_0$ は

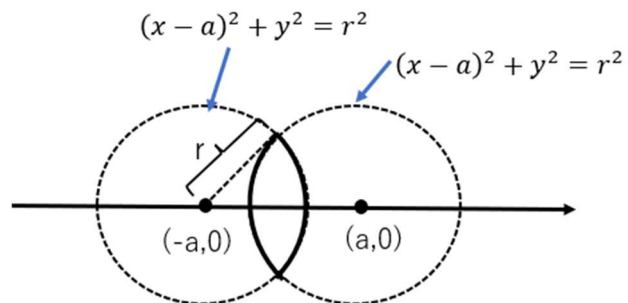


図 2.3 凹レンズ面の数式化

$$\mathbf{d}_0 = (p_0 - p_1, q_0 - q_1) \quad (2.8)$$

$F(x, y) = (x - a)^2 + y^2 - r^2$ とし、式(1.3)(1.4)と同様に法線ベクトルを求める。

$$\begin{aligned}\nabla F &= \left(\frac{\partial F}{\partial x}, \frac{\partial F}{\partial y} \right) = (2(x - a), 2y) \\ &= 2(x - a, y)\end{aligned}\quad (2.9)$$

よって、法線ベクトル $\mathbf{n}$ は、

$$\mathbf{n} = (p_1 - a, q_1) \quad (2.10)$$

凸レンズを通る光路ベクトルは式(2.7)の通りである。これを直線の式にすると、以下の式となる。

$$(x, y) = (p_1, q_1) + t(d_{1x}, d_{1y}) \quad (2.11)$$

次に、凸レンズから出る光路を考える。図 2.5 の点

$P_2(p_2, q_2)$ を求める。これは、式(2.11)と円弧

$(x + a)^2 + y^2 = r^2$ との交点なので、式(2.11)を円弧の式に代入して

$$\begin{aligned}(p_1 + td_{1x} + a)^2 + (q_1 + td_{1y})^2 &= r^2 \\ At^2 + Bt + C &= 0\end{aligned}\quad (2.12)$$

ただし

$$\begin{aligned}A &= d_{1x}^2 + d_{1y}^2, \\ B &= 2\{d_{1x}(p_1 + a) + q_1 d_{1y}\}, \\ C &= (p_1 + a)^2 + q_1^2 - r^2\end{aligned}$$

式(2.12)について、 t について解き $t > 0$ の解を式(2.11)に代入した時の (x, y) が $P_2(p_2, q_2)$ となる。入射ベクトル $\mathbf{d}_1'$ を図 2.5 のようにとると、成分表示は以下ようになる。

$$\mathbf{d}_1' = (p_1 - p_2, q_1 - q_2) \quad (2.13)$$

法線ベクトル $\mathbf{n}'$ は、 $F'(x, y) = (x + a)^2 + y^2 - r^2$ にできる法線であるため、

$$\nabla F' = \left(\frac{\partial F'}{\partial x}, \frac{\partial F'}{\partial y} \right) = (2(x + a), 2y) = 2((x + a), y) \quad (2.14)$$

よって、法線ベクトル $\mathbf{n}'$ は以下ようになる。

$$\mathbf{n}' = (p_2 + a, q_2) \quad (2.14)$$

凸レンズから出るベクトル $\mathbf{d}_2$ は式(2.7)と同様に考えて以下ようになる。

$$\mathbf{d}_2 = -\frac{\eta_1}{\eta_0}(\mathbf{d}_1 - (\mathbf{d}_1 \cdot \hat{\mathbf{n}}')\hat{\mathbf{n}}') - \hat{\mathbf{n}}' \sqrt{1 - \left(\frac{\eta_1}{\eta_0}\right)^2 (1 - (\hat{\mathbf{d}}_1 \cdot \hat{\mathbf{n}}')^2)} \quad (2.15)$$

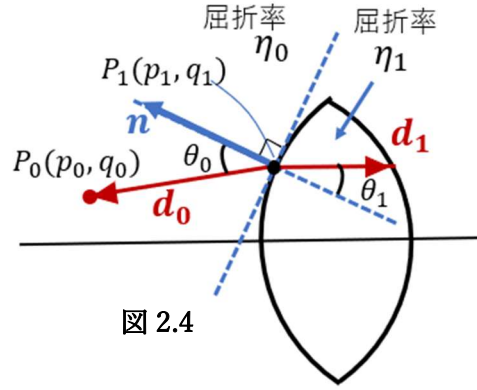


図 2.4

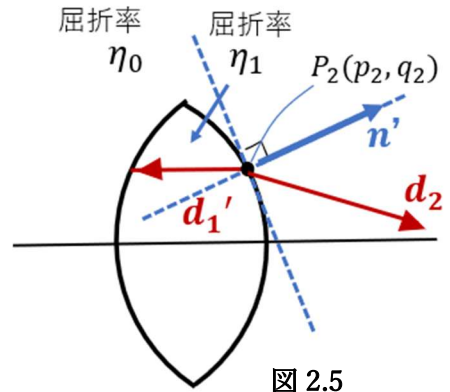
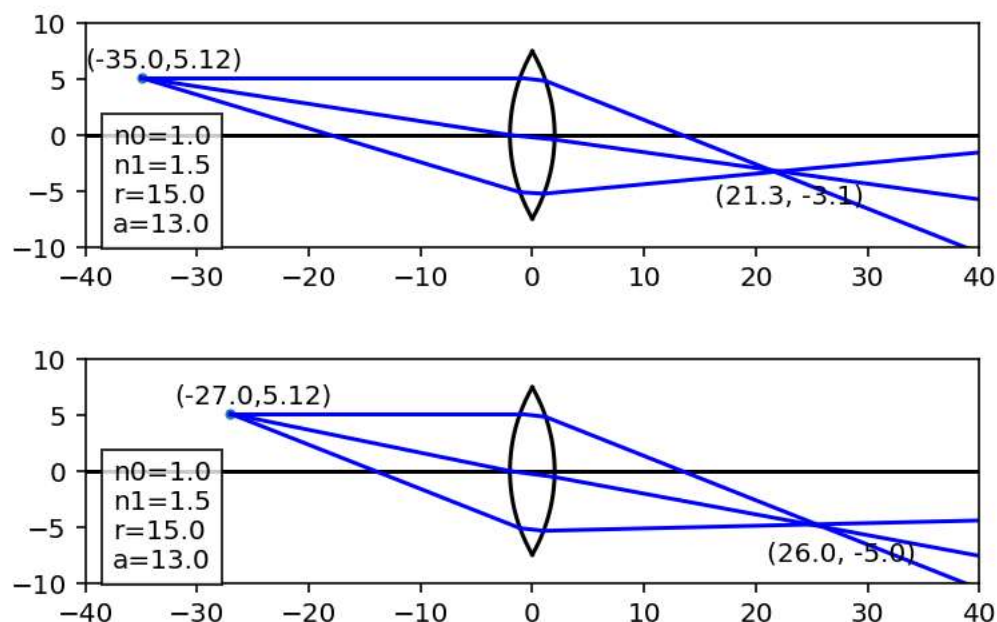


図 2.5

計算結果を以下に示す。

●点光源が凸レンズに近い場合と遠い場合での比較

点光源の位置が(-35,5.12)、(-27,5.12)の時の計算結果を以下に示す。



点光源が凸レンズから遠い場合と近い場合で比較をすると、点光源が凸レンズに近づくほど、凸レンズから遠ざかることがわかる。

焦点には以下の法則が成り立つが、これは、物体距離 s が小さくなるほど、像距離が大きくなることを示し、計算結果と合うことがわかる。

$$\frac{1}{f} = \frac{1}{s} + \frac{1}{s'} \quad (2.16)$$

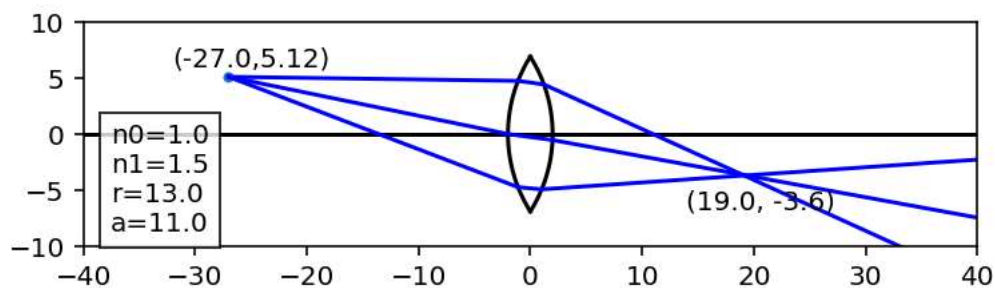
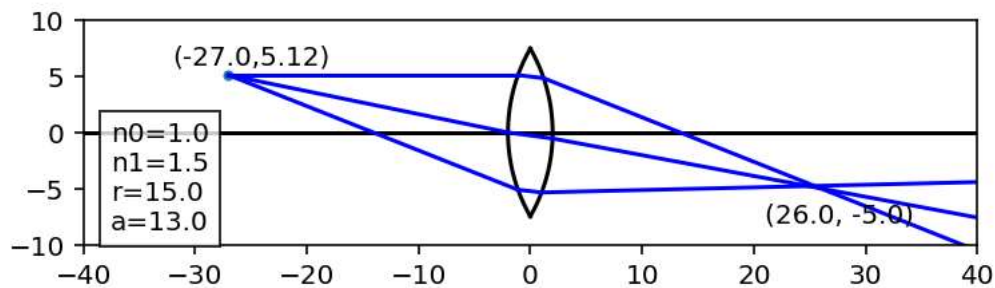
ただし、 f : 焦点距離

s : 物体距離(レンズから点光源まで)

s' : 像距離

●凸レンズの曲率半径による比較

凸レンズの曲率半径 r が $r=13$ と $r=11$ の場合の計算結果を以下に示す。



曲率半径が小さいほど、点光源の位置が近づいているのがわかる。

曲率半径と焦点距離には以下の関係が成り立ち、これは、レンズの曲率半径 R が小さくなるほど、焦点距離 f が小さくなることを示し、計算結果と合うことがわかる。

$$f = \frac{R}{2(n-1)}$$

ただし、 f : 焦点距離

n : レンズの屈折率

R : レンズの曲率半径

<凹面鏡の計算プログラム>

```
import numpy as np
import matplotlib.pyplot as plt

d = 3
l = 8

alpha = l**2
beta = l**2-d**2
theta = np.linspace(-np.pi/2,np.pi/2,400)
x = np.sqrt(alpha)*np.cos(theta)
y = np.sqrt(beta)*np.sin(theta)
plt.plot(x,y,'k-')
plt.axis([-10,12,-6,6])
plt.xticks(np.arange(-10, 14, 2))
plt.yticks(np.arange(-8, 10, 2))

a0 = -1*d
```

```

b0 = 0

theta0_list = np.linspace(-2*np.pi/6, 2*np.pi/6, 5)
p0 = np.sqrt(alpha) * np.cos(theta0_list)
q0 = np.sqrt(beta) * np.sin(theta0_list)

for px, qy in zip(p0, q0):
    plt.plot([a0, px], [b0, qy], 'b-')
    nx = px/alpha
    ny = qy/beta
    norm = np.sqrt(nx**2+ny**2)
    nx_hat = nx/norm
    ny_hat = ny/norm
    dx = px-a0
    dy = qy-b0
    dot = dx*nx_hat+dy*ny_hat
    rx = dx-2*dot*nx_hat
    ry = dy-2*dot*ny_hat
    t = np.linspace(0, 5, 100)
    rx_line = px + t * rx
    ry_line = qy + t * ry
    plt.plot(rx_line, ry_line, 'c-')

plt.scatter(a0, b0, color='black')
plt.text(d+0.5, 0.5, f"({d}, 0)")
plt.scatter(d, 0, color='black')
plt.text(a0-2, b0+0.5, f"({a0}, {b0})")

plt.text(
    0.05, 0.95,
    f"d = {d} \n l = {l}",
    transform=plt.gca().transAxes,
    verticalalignment='top',
    bbox=dict(facecolor='white', alpha=0.8)
)
plt.plot([], [], color='blue', label='Incident ray')
plt.plot([], [], color='cyan', label='Reflected ray')
plt.legend()

plt.gca().set_aspect('equal', adjustable='box')
plt.show()

```

<凸レンズ計算プログラム>

```
import numpy as np
import matplotlib.pyplot as plt
#凸レンズを用意
a = 11.0
r = 13.0

theta = np.arccos(a/r)
theta_list=np.linspace(-1*theta,theta,400)
x0 = r*np.cos(theta_list)-a
y0 = r*np.sin(theta_list)
plt.plot(x0,y0,'k-')
x1 = r*np.cos(np.pi-theta_list)+a
y1 = r*np.sin(np.pi+theta_list)
plt.plot(x1,y1,'k-')
plt.axis([-40,40,-10,10])
plt.plot([-40.0,40.0],[0.0,0.0],'k-')
plt.gca().set_aspect('equal',adjustable='box')
#点光源の位置
p0=-27.0
q0=5.12
#r*np.sin(np.pi-theta*2/3)

plt.plot(p0,q0,".")
plt.text(p0-5,q0+1,f"({p0:.1f},{q0:.2f})")

phi_list=np.linspace(np.pi-theta*2/3,np.pi+theta*2/3,3)
p1 = r*np.cos(phi_list)+a
q1 = r*np.sin(phi_list)

n0 = 1.0
n1 = 1.5
#光路の交点を求める用に、それぞれの点を保管する箱を用意
lines = []
#凸レンズから出る光の始点の座標を保管する用の箱を用意
p2_list=[]
q2_list=[]

for p1x,q1y in zip(p1,q1):
    plt.plot([p0,p1x],[q0,q1y],color='blue')
    d0x = p0-p1x
    d0y = q0-q1y
    d0norm = np.sqrt(d0x**2+d0y**2)
    d0x_hat = d0x/d0norm
    d0y_hat = d0y/d0norm
    nx = p1x-a
    ny = q1y
    nnorm = np.sqrt(nx**2+ny**2)
    nx_hat = nx/nnorm
    ny_hat = ny/nnorm
    d0dot = d0x_hat*nx_hat+d0y_hat*ny_hat
    d1x = (-1*n0/n1*(d0x_hat-d0dot*nx_hat)-
           nx_hat*np.sqrt(1-((n0/n1)**2)
                        *(1-d0dot**2)))
```

```

d1y = (-1*n0/n1*(d0y_hat-d0dot*ny_hat)-
        ny_hat*np.sqrt(1-((n0/n1)**2)
                        *(1-d0dot**2)))
t1_line = np.linspace(0,20,100)
d1x_line = p1x+t1_line*d1x
d1y_line = q1y+t1_line*d1y
mask1 = (d1x_line+a)**2+d1y_line**2<r**2
plt.plot(d1x_line[mask1],d1y_line[mask1],
         color='blue')
A = d1x**2+d1y**2
B = 2*(d1x*(p1x+a)+q1y*d1y)
C = (p1x+a)**2+q1y**2-r**2
#判別式
D = B**2-4*A*C
if(D>=0):
    t1_1 = (-1*B+np.sqrt(B**2-4*A*C))/(2*A)
    t1_2 = (-1*B-np.sqrt(B**2-4*A*C))/(2*A)
    t1 = min(t for t in [t1_1,t1_2] if t>0)
p2 = p1x+t1*d1x
q2 = q1y+t1*d1y

p2_list.append(p2)
q2_list.append(q2)

d1x_prime = p1x-p2
d1y_prime = q1y-q2
d1norm = np.sqrt(d1x_prime**2+d1y_prime**2)
d1x_hat = d1x_prime/d1norm
d1y_hat = d1y_prime/d1norm
nx_prime = -(p2+a)
ny_prime = -q2
nprimenorm = np.sqrt(nx_prime**2+ny_prime**2)
nxprime_hat = nx_prime/nprimenorm
nyprime_hat = ny_prime/nprimenorm
d1dot = d1x_hat*nxprime_hat+d1y_hat*nyprime_hat
d2x = (-1*n1/n0*(d1x_hat-d1dot*nxprime_hat)-
        nxprime_hat*np.sqrt(1-((n1/n0)**2)
                        *(1-d1dot**2)))
d2y = (-1*n1/n0*(d1y_hat-d1dot*nyprime_hat)-
        nyprime_hat*np.sqrt(1-((n1/n0)**2)
                        *(1-d1dot**2)))
t2_line = np.linspace(0,50,100)
d2x_line = p2+t2_line*d2x
d2y_line = q2+t2_line*d2y
plt.plot(d2x_line,d2y_line,color='blue')
lines.append((p2,q2,d2x,d2y))
#光路の交点を求める
(x1,y1,v1x,v1y) = lines[0]
(x2,y2,v2x,v2y) = lines[1]
E = np.array([[v1x,-v2x],
              [v1y,-v2y]])
F = np.array([x2-x1,y2-y1])
t, s = np.linalg.solve(E, F)
x = x1 + t*v1x

```

```

y = y1 + t*v1y
plt.plot(x,y,'k-')
plt.text(x-5, y-3, f"({x:.1f}, {y:.1f})")

#座標を表示
#for px_1, py_1 in zip(p1, q1):
#    plt.text(px_1-15, py_1+2,
#             f"({px_1:.3f}, {py_1:.3f})")

#for px_2, py_2 in zip(p2_list, q2_list):
#    plt.text(px_2+1, py_2+2,
#             f"({px_2:.3f}, {py_2:.3f})")

plt.text(
    0.03,0.55,
    f"n0={n0}✕nn1={n1}✕nr={r}✕na={a}",
    transform = plt.gca().transAxes,
    verticalalignment='top',
    bbox = dict(facecolor = 'white',alpha=0.8)
)

plt.show()

```